

Hacking de Binários em Hexadecimal

Uma das técnicas mais conhecidas e usadas para quebrar aplicações e subvertê-las é modificando diretamente o binário gerado. Uma forma de fazer isso é usando um simples editor hexadecimal para modificar o binário sem, necessariamente, alterar o seu código fonte.

Essa técnica é muito utilizada na tradução de programas e jogos de computadores (e, para algumas mentes não bem intencionadas, outros fins). Embora seja uma técnica simples, os binários costumam usar estratégias para ocultar as informações como compressão (Huffman, LZ77, etc), tabelas e ponteiros. Isso torna a aplicação geral da técnica muito mais complexa.

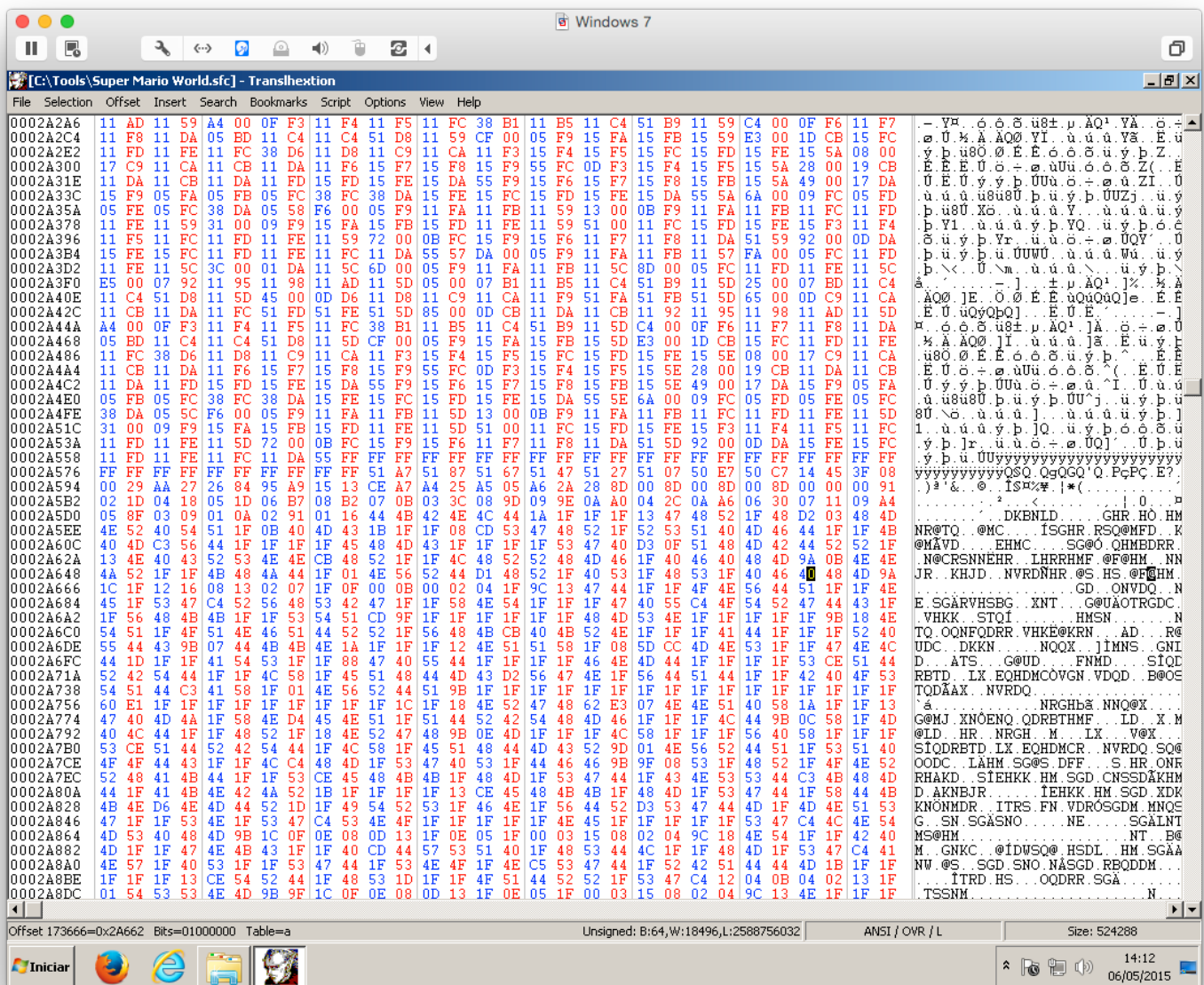
O objetivo deste trabalho é subverter uma ROM (o *dump* de um cartucho de video game) do jogo Super Mario Bros de Super Nintendo para traduzir as mensagens do jogo. A imagem a seguir mostra um exemplo de mensagem:



Trabalho de Segurança de Redes

A mensagem na imagem é: *Welcome! This is Dinosaur Land. In this strange land we find that Princess Toadstool is missing again! Looks like Bowser is at it again!*

Se abrirmos a ROM (binário) do jogo em um editor hexadecimal como o **Transhextion16c**, é possível abrir o ROM e ver o seu conteúdo:

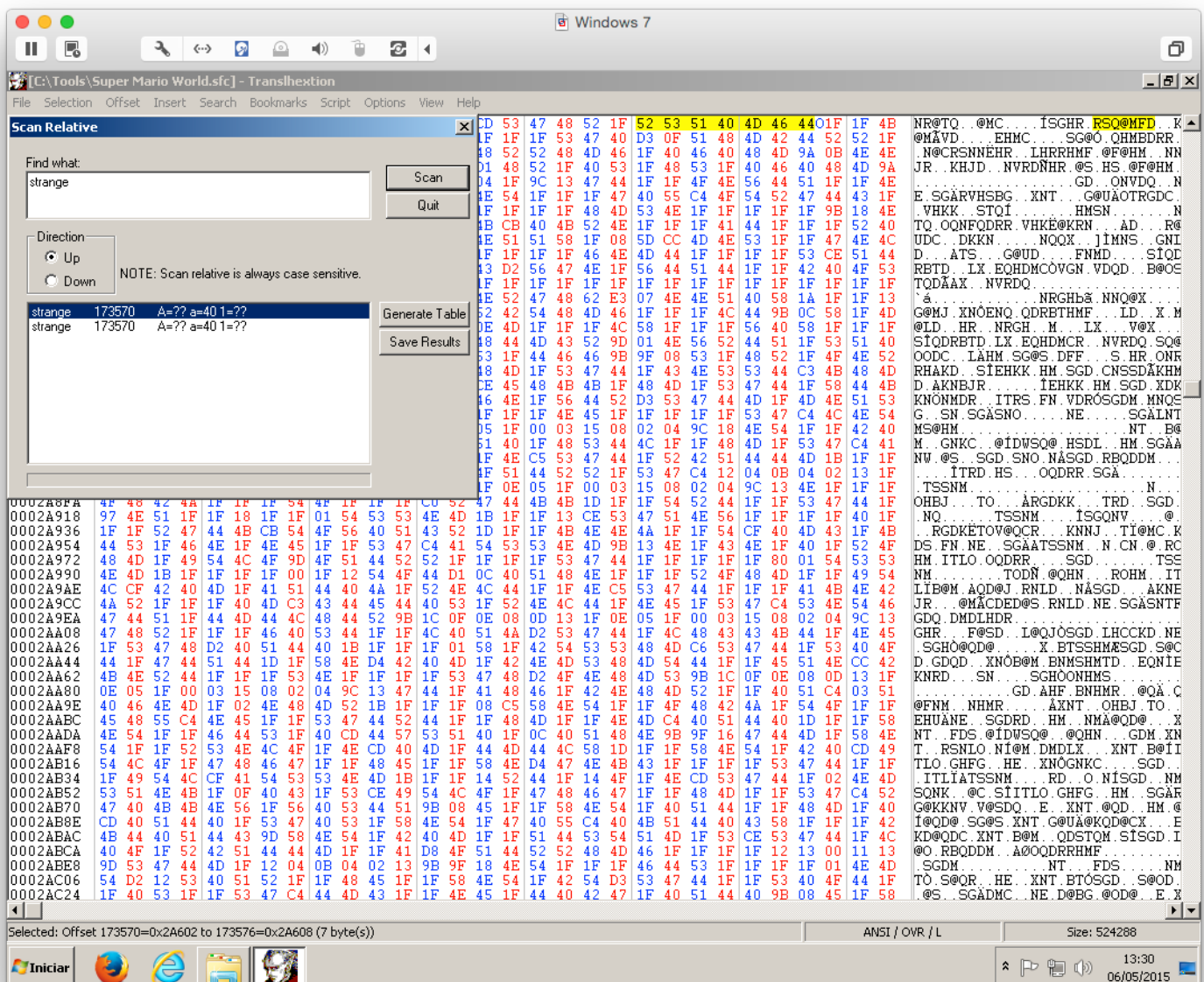


Da para notar que está totalmente incompreensível. Isso é verdade porque não conseguimos interpretar diretamente os valores hexadecimais para letras e caixas de texto de forma simples.

Trabalho de Segurança de Redes

No caso dessa ROM em especial, para “enxergarmos” basta criarmos uma tabela que relacione os valores em hexadecimal com as letras correspondentes, ou seja, mapear a tabela ASCII com valores diferentes dos já conhecidos.

O Translhex16c possui um mecanismo de busca relativa que busca usando a relação entre a distância das letras de uma palavra. Então, para acharmos uma “relação”, basta encontrarmos quais cadeias hexadecimais casam com os “valores” encontrados na ROM. Realizando uma busca relativa vemos que é possível encontrar uma relação entre valores em hexa e letras:



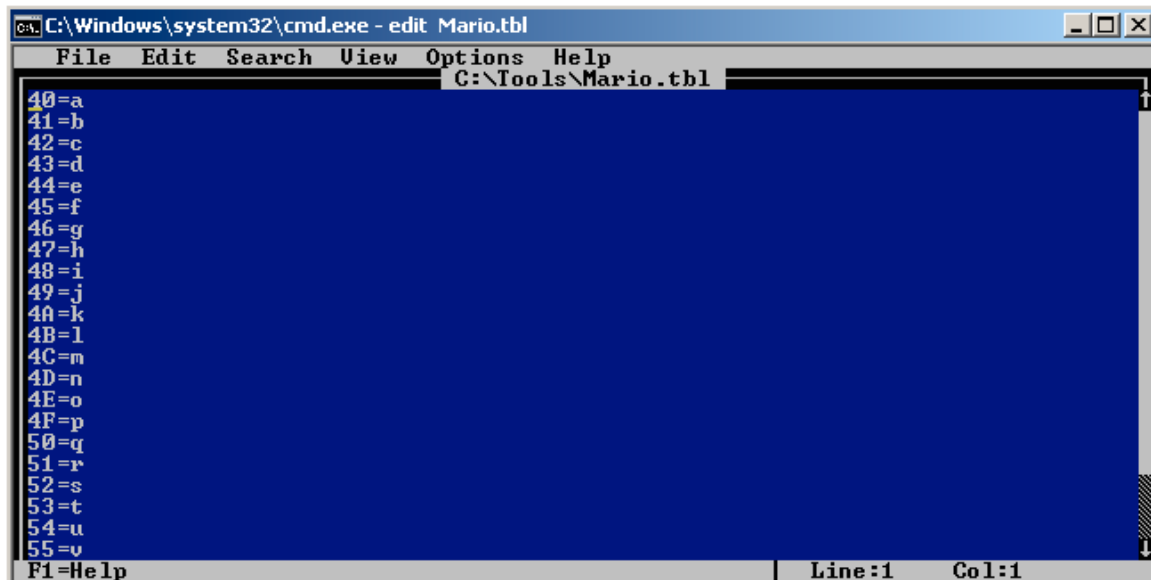
The screenshot shows the Translhex16c application window. The title bar reads "[C:\Tools\Super Mario World.sfc] - Translhex16c". The menu bar includes File, Selection, Offset, Insert, Search, Bookmarks, Script, Options, View, and Help. The toolbar contains icons for various functions. The main window is divided into several sections:

- Scan Relative:** A section on the left with a "Find what:" field containing "strange". Below it are "Scan" and "Quit" buttons. There are also "Generate Table" and "Save Results" buttons.
- Direction:** Radio buttons for "Up" and "Down". A note states: "NOTE: Scan relative is always case sensitive."
- Search Results:** A table showing the search results. The first two rows are:

Address	Hex Value	ASCII
strange	173570	A=?? a=40 1=??
strange	173570	A=?? a=40 1=??
- ROM Data:** A large table of hexadecimal data from the ROM, with columns for address, hex value, and ASCII. The data is displayed in a grid format.
- Status Bar:** At the bottom, it shows "Selected: Offset 173570=0x2A602 to 173576=0x2A608 (7 byte(s))", "ANSI / OVR / L", and "Size: 524288".

Dessa forma podemos gerar a tabela relativa usando o botão: *Generate Table*.

E, dessa forma, montar uma tabela de “equivalência”.



```
C:\Windows\system32\cmd.exe - edit Mario.tbl
File Edit Search View Options Help
C:\Tools\Mario.tbl
40=a
41=b
42=c
43=d
44=e
45=f
46=g
47=h
48=i
49=j
4A=k
4B=l
4C=m
4D=n
4E=o
4F=p
50=q
51=r
52=s
53=t
54=u
55=v
F1=Help | Line:1 Col:1
```

Obviamente a tabela acima está incompleta e não está mapeando caracteres como SPACE (espaço) ou as letras maiúsculas, mas já serve como um exemplo do que é necessário fazer. O seu trabalho é escrever programas (em C/C++, Python ou Java) para:

Parte 1:

- Criar uma ferramenta (*finder.exe*) que receba uma *string* e retorne a posição na ROM (*offset*) e os valores hexadecimais encontrados.
- Criar uma ferramenta (*extractor.exe*) que leia o ROM e extraia os textos do jogo de tal forma que seja possível modificar a sua extração (traduzindo os textos) para futura reinserção.
- Criar uma ferramenta (*injector.exe*) que injete os textos modificados pela ferramenta anterior diretamente na ROM.

Parte 2:

- Traduzir a mensagem inicial do jogo no seu arquivo extraído.
- Inserir o texto na ROM editando diretamente o binário.
- Escrever um relatório de no máximo uma página explicando como foi criada a tabela e as estratégias adotadas para extrair e inserir o texto no binário.

Para testar se a sua *ROMhacking* teve sucesso use um emulador de SNES como SNES9x (possui versões para Windows, Linux, Android). No MacOSX é possível usar o OpenEmu (que usa o motor do SNES9x).

O trabalho é individual e a entrega deverá ser feita no Moodle da disciplina. Todos os programas e o relatório devem ser enviados em um único arquivo .ZIP. Uma observação importante: *O jogo possui mais de uma tabela ASCII para representar caracteres!*

Apêndice

Apenas a título de curiosidade, seguem algumas extrações feitas diretamente da ROM usando um programa Python que usa um dicionário para mapear a DTL:

Texto 1: Welcome! This is Dinosaur Land. In this strange land we find that Princess Toadstool is missing again! Looks like Bowser is at it again!



E o resultado da extração usando uma tabela previamente criada:

```
Brivaldos-Mac-mini:Src condecor$ python Viewer.py SMario.sfc | grep -A6 Welcome
C91BWelcome! This is
Dinosaur Land. Inthi
s strange landwe
find thatPrincess
Toadstoolis missing
again!Looks like Bow
seris at it again!1C S
Brivaldos-Mac-mini:Src condecor$
```

Texto 2: *Hooray! Thank you for rescuing me. My name is Yoshi. On my way to rescue my friends, Bowser trapped me in that egg.*



E o resultado da extração usando uma tabela previamente criada:

```
Brivaldos-Mac-mini:Src conector$ python Viewer.py SMario.sfc | grep -A5 Hooray
shi62E3Hooray! Thank y
oufor rescuing me.M
y name is Yoshi.On
my way torescue
my friends,Bowser tra
pped mein that egg.9F
Brivaldos-Mac-mini:Src conector$
```